

Firelight A-1

Security Audit

July 29, 2026

Version 1.0.0

Presented by [0xMacro](#)

[Original online report](#)

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Firelight's smart contract code as found in the section titled 'Source Code'. The security audit was performed by the Macro security team on June 26th to July 13th 2026.

The purpose of this audit is to review the source code of certain Firelight Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Medium	2	-	-	2
Low	4	1	-	3
Informational	11	1	-	10
Gas Optimization	1	-	-	1

Firelight was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Firelight team.
- Available documentation in the repository.

Trust Model, Assumptions, and Accepted Risks (TMAAR)

Firelight is an on-chain cover protocol on Flare. For the MVP it is, by design, an operator run underwriting desk with on-chain settlement rails: off-chain actors sell cover, assess incidents, and execute payouts, while the contracts enforce per action bounds and hold funds. This document outlines what each participant must trust and what happens if a trusted party is compromised or errs. The privileged powers listed here are conditions for correct operation, not vulnerabilities in themselves.

The v2 contracts in scope are not yet deployed, so their role holders below are the intended holders described by the team. The legacy V1 predeposit vault, however, is live on Flare mainnet and holds real capital; its control structure was verified on-chain and is described in the On-Chain Verification section, and the audited v2 code is intended to upgrade it.

Two participant classes carry very different exposure:

- **Collateral providers (XRP stakers)** are protected by code. Slashing is bounded and pro rata, and their downside can never exceed their stake. Their residual exposure is governance and the solvency parameters.
- **Cover buyers** are protected mainly by permissioned users executing based off legal or business agreements, not by code. Their orders are authored by the operator, their claims are assessed by the operator and approved by a consortium, and their payouts are distributed manually. On-chain they only grant a premium approval and hold a receipt NFT.

External Infrastructure

Flare FTSO v2 (via the Flare Contract Registry)

Trusted for the integrity of the vault asset price (FXRP in USD), which drives both cover capacity and the vault of payouts. The FtsoChainlinkAdapter reads the feed live through a static call and validates positivity and freshness before use, so a bad or zero read fails closed rather than mispricing.

- The adapter works only while the FTSO feed is callable without a fee, which is the current default. If Flare enables fees on the wrapped feed, every price read reverts and both cover commitment and vault payouts halt until the operators migrate to a fee adapter. This is a live external dependency for the whole pricing surface.
- FTSO integrity is a Flare infrastructure assumption outside this protocol's control. A manipulated or degraded feed mis sizes capacity and payouts. Flare is an L1, so there is no sequencer liveness concern.

First Loss Buffer custody wallet

A stablecoin reserve that both contributes to cover capacity and pays first in the claim payout waterfall. It is an external Firelight operated wallet configured per period, not an escrow held by the protocol.

- Capacity reads the wallet balance live, and the waterfall pulls from it via a set approval. The protocol trusts this wallet to remain funded and approved. If it is funded at commitment and then drawn down, sold cover becomes underbacked. If it holds a balance but has not approved the IncidentManager, incident approval reverts.
- The buffer is assumed to be a stablecoin pegged to USD. Its USD value is not oracle checked on-chain (see Accepted Risks).

Premium Collection Address and daily conversion service

Receives all premiums and runs a daily strategy that converts them into the vault asset and streams the result into the vault as staker yield. Trusted to collect premium honestly and return it as yield. A misconfigured or compromised collector could divert or steal staker yield. Operationally isolated from payout flows.

Protocol Governance

ProxyAdmin owner and DEFAULT_ADMIN_ROLE (Firelight)

Highest trust level. All contracts sit behind Transparent proxies, the ProxyAdmin owner can replace the logic of every contract, which overrides every restriction and invariant of the protocol. DEFAULT_ADMIN_ROLE on each contract grants and revokes all other roles and can assign itself any operational power below.

For the live V1 vault, this authority is verified on-chain (see On-Chain Verification): the ProxyAdmin owner and the vault DEFAULT_ADMIN_ROLE are the same 2 of 2 Safe, whose two members are themselves a 6 of 11 and a 3 of 4 Safe. This is a strong dual control structure.

Protocol Roles

The roles below are operational trust assumptions. Holders are assumed to act within established policies. If a holder is compromised or acts maliciously, the impact is as described.

Curator (Sentora): CURATOR_ROLE and ALLOCATOR_ROLE on the CoverOrderAllocator, CURATOR_ROLE on the IncidentManager

The operational heart of the MVP. The curator creates cover orders on behalf of clients and sets every order parameter, commits the off-chain matching result and settles it (charging premium and minting cover NFTs), and creates incidents, chooses their capture timestamp, and publishes the proposed loss schedule.

Trusted to author orders on agreed terms, match honestly, and report incidents and losses truthfully. If compromised, it can charge buyers for unwanted or mispriced cover or propose fabricated incidents. Premium flows to the collector rather than the curator, so premium extraction is theft only if the same party controls the collector or holds the allocator ADMIN_ROLE.

Risk Consortium Safe: ASSESSMENT_APPROVER_ROLE, ASSESSMENT_REJECTER_ROLE, and INCIDENT_INVALIDATOR_ROLE on the IncidentManager

Intended as a multisign with a 5 member consortium and a 3-out-of-5 quorum threshold. The payout authority and the primary on-chain check on the operational proposals of incidents and payouts.

Trusted to validate the curator's loss schedule (both coverage validity and payout accuracy) before approving. If it colludes with the curator or approves without genuine review, it can unfairly slash the buffer and vault assets. Its independence from the curator is the single most important assumption in the claim flow. Note that all three claim-side powers live in this one Safe.

PAYOUT_ROLE and INCIDENT_ROLE on the FirelightVault

Operational roles in FirelightVault intended to be held by the IncidentManager contract, not by any EOA. PAYOUT_ROLE is the only path by which vault assets leave to the cover side, and INCIDENT_ROLE freezes deposits during an open incident. The vault trusts the caller entirely for legitimacy and enforces only per-call bounds. Because these roles are contract held, misuse requires the curator and consortium path above rather than a single key. It is a deployment invariant that these roles are held only by the IncidentManager.

MINTER_ROLE on the CoverNFT

Held by the allocator contract, which mints one cover NFT per settled order to the buyer. It is a deployment invariant that this role is held only by the allocator; otherwise arbitrary token ids or recipients could be minted.

Configuration and value setting roles (Firelight)

- **Allocator** CONFIG_ADMIN_ROLE sets the solvency parameters (leverage, minCAR, first loss buffer token and wallet, divergence tolerance), the settlement grace period, markets, and concentration caps. Misuse can over-leverage the protocol (see M-1, L-4) or strand orders. Highest impact configuration role.
- **Allocator** ADMIN_ROLE sets the premium collector and the capacity oracle. Redirecting the collector diverts all premiums.
- **IncidentManager** PAYOUT_ADMIN_ROLE sets the single payout receiver that receives all buffer and vault payouts. Redirecting it captures payouts before the manual distribution step.
- **IncidentManager and adapter oracle roles** (PRICE_FEED_ADMIN_ROLE, FEED_ADMIN_ROLE) change the payout oracle and rotate the wrapped FTSO feed. Feed rotation has no timelock. The allocator and IncidentManager price feeds are configured independently in code; both are expected to reference the single FTSO FXRP/USD adapter, which is a deployment configuration assumption verified in the checklist below rather than an enforced invariant.
- **Vault** BLOCKLIST_ROLE with RESCUER_ROLE can blocklist any account and move its shares and pending withdrawals to an address of choice. This is the standard compliance rescue pattern and is a targeted confiscation power over a staker's principal.
- **Vault** PAUSE_ROLE, DEPOSIT_LIMIT_UPDATE_ROLE, PERIOD_CONFIGURATION_UPDATE_ROLE, PAYOUT_ALLOWLIST_ROLE; **CoverNFT** PAUSER_ROLE and URI_MANAGER_ROLE; **rewarder** DISTRIBUTOR_ROLE and SWEEPER_ROLE are lower impact operational levers (pause, deposit throttling, period cadence, payout allowlist, NFT metadata base URI, yield injection, stray token recovery). Pausing the CoverNFT also halts settlement. URI_MANAGER_ROLE is cosmetic only: it sets the CoverNFT metadata base URI and touches no funds or liveness.

Firelight Foundation Multisig (off-chain distributor)

Receives the pooled payout at the single payout receiver and distributes each beneficiary's pro rata share manually, off-chain for this MVP version. Cover buyers trust this party to forward their share correctly. This is a custodial step at the edge of the system.

On-Chain Verification

The values below were read from live calls against the Flare mainnet via a public node RPC on audit dates (July 10, 2026), with the ERC1967 implementation and admin slots read directly. Only the legacy V1 predeposit vault is deployed today. The v2 contracts in scope are not yet deployed, so their role holders remain intended as described above. This section characterizes the live control surface that custodies predeposit capital now, and that will execute the upgrade to v2.

Governance structure

The V1 vault is a Transparent proxy. Its upgrade authority (the ProxyAdmin owner) and its `DEFAULT_ADMIN_ROLE` are the same Gnosis Safe, a 2 of 2. The two members of that Safe are not individual keys; each is itself a Gnosis Safe:

- Member A is a 6 of 11 Safe.
- Member B is a 3 of 4 Safe.

Changing the vault logic or granting any vault role therefore requires both member Safes to approve, which in turn requires a 6 of 11 quorum in one organization and a 3 of 4 quorum in the other. This is a dual control structure between two multisig parties, each with internal redundancy, rather than a thin two key setup.

Address Legend

Label	Address	Type	Description
Root Safe	0xf811e83b45d8de67858efa3ac1202a8b46b8d0b3	Gnosis Safe, 2 of 2	Owns the ProxyAdmin and holds <code>DEFAULT_ADMIN_ROLE</code> on the vault
Member Safe A	0x78547dda5c18db47bb8ecd1d3c368955b513f355	Gnosis Safe, 6 of 11	One of the two required members of the root Safe
Member Safe B	0xb75e26b07253172348c34a70802b09f9ed735438	Gnosis Safe, 3 of 4	One of the two required members of the root Safe
ProxyAdmin	0x15f89292cef63405a53ab806966681ecb430df85	ProxyAdmin	Transparent proxy admin, owned by the root Safe
Vault asset	0xad552a648c74d49e10027ab8a618a3ad4901c5be	ERC20, FXRP, 6 decimals	Underlying collateral
Operations Safe	0x189993f3B53284c7e5C02b058EFC6618F78D0cEA	Gnosis Safe, 1 of 1	Holds <code>PAUSE</code> , <code>BLOCKLIST</code> , and <code>PERIOD_CONFIGURATION_UPDATE</code>
Deposit limit Safe	0x877EC24ad923De4D22bA24eB1A6A3564AbC94564	Gnosis Safe, 1 of 1	Holds <code>DEPOSIT_LIMIT_UPDATE</code>
Rescuer Safe	0xC115757266b902d4b1284eA24DBbE557C15E963E	Gnosis Safe, 1 of 1	Holds <code>RESCUER</code>

Contract Registry

Contract	Proxy	Implementation
V1 predeposit vault	0x4C18Ff3C89632c3Dd62E796c0aFA5c07c4c1B2b3	0x70ccf1bee0c1217069fe74083ca71af7bcd7fb76

Observed State

Item	Value	Note
contractVersion()	1	Confirms the pre v2 predeposit vault
asset()	FXRP, 6 decimals	Underlying collateral
totalAssets()	approximately 60.16 million FXRP	Real predeposit capital held today
DEFAULT_ADMIN_ROLE holder	Root Safe (2 of 2)	The two member Safes do not hold the role individually

Role Matrix

Role	Holder	Type
DEFAULT_ADMIN_ROLE	Root Safe 0xf811e83b45d8de67858efa3ac1202a8b46b8d0b3	2 of 2 (members are a 6 of 11 and a 3 of 4 Safe)
PAUSE_ROLE	0x189993f3B53284c7e5C02b058EFC6618F78D0cEA	Gnosis Safe, 1 of 1
BLOCKLIST_ROLE	0x189993f3B53284c7e5C02b058EFC6618F78D0cEA	Gnosis Safe, 1 of 1
PERIOD_CONFIGURATION_UPDATE_ROLE	0x189993f3B53284c7e5C02b058EFC6618F78D0cEA	Gnosis Safe, 1 of 1
DEPOSIT_LIMIT_UPDATE_ROLE	0x877EC24ad923De4D22bA24eB1A6A3564AbC94564	Gnosis Safe, 1 of 1
RESCUER_ROLE	0xC115757266b902d4b1284eA24DBbE557C15E963E	Gnosis Safe, 1 of 1

The original deployer 0x3e6F51038D42E3Dd34b234813ec32bd6074E4Ee7 held DEFAULT_ADMIN_ROLE at genesis and was revoked once the root Safe was granted it, so no deployer key retains admin privileges.

Accepted and Known Risks

These are conditions the team has reviewed and accepted for the MVP, or documented as known issues. They are ordered by blast radius. Several rest on the same off chain operator and should be revisited before a mainnet deployment that holds real staker capital.

Aggregate claims are not capped on-chain against cover sold (known issue, observed on testnet)

The contracts do not track how much of a given cover has already been claimed within a period. Each assessed loss is checked only against a cover market's fixed allocation, and the duplicate check is scoped to a single incident and round, so two incidents in the same period can each claim the full allocation for the same cover. The team flags this to auditors in the specification, and their own testnet paid out more than 100 percent of notional across incidents in a single period. It requires a curator and the consortium to approve duplicate or excessive claims, or an operational oversight.

The sole control today is off-chain review by the consortium. Impact if it recurs is staker loss beyond underwritten exposure. The team's planned fix is on-chain per coverage tracking (a `remainingCoverage` decrement on approval). This is accepted for the MVP.

First loss buffer is counted and paid from an unescrowed external wallet (trust assumption for MVP)

Capacity counts, and the waterfall draws from, the live balance of an external address. It can be funded to inflate capacity at commitment and then withdrawn, and its funding and approval state is a liveness dependency for claim approval. Accepted for the MVP under the assumption that the buffer wallet is honestly managed by the operator.

Cover orders are authored entirely by the operator, gated only by the buyer's approval (accepted for MVP)

For the MVP, the curator places orders on behalf of clients under an existing off-chain business relationship, and the buyer does not interact directly with the protocol. The only on-chain consent gate is the buyer's premium approval, which bounds premium value at risk but not the order parameters. Because coverage can be renewed per period with a fresh premium pull each time, recurring buyers overapproving their recurring premium for more to avoid constantly managing their allowances are more exposed to the protocol discretion. Accepted as an explicit trust assumption for the MVP, with buyers creating their own orders post MVP. Recommendations: bound the per-order premium on-chain (a maximum rate), keep the curator and the collector setting role separate, and adopt signed order consent (EIP712 with a per-buyer nonce) for the post-MVP self-serve path.

First loss buffer is treated as exactly USD pegged (accepted, documented)

The buffer's USD value is derived solely via decimal conversion, with no oracle. A depeg of the buffer stablecoin overcounts capacity and, on the payout side, credits the buffer at par, so the depeg loss is routed to the beneficiary while the vault is shielded. Accepted given a standard, high-trust stablecoin is intended to be used and clear documentation. The assumption is only as strong as the configured buffer token, so a change of that token via the capacity configuration should be reviewed.

New deposits can capture a pending reward distribution (accepted)

Because rewards are injected by raising the vault redemption value, a deposit placed just before a distribution shares in yield it did not earn, diluting existing stakers for that cycle. It is mitigated by the withdrawal delay and by incident risk, since the new capital cannot exit immediately and remains exposed for the period.

Incident approval is FIFO by capture time and depends on operators resolving earlier incidents

In `IncidentManager`, `approveCurrentAssessment` must target the earliest non-terminal incident in the payout window, so a later incident cannot be approved while an earlier one is still `OPEN`, `CONFIRMED`, or under evaluation without a submitted round. This capture time priority is intentional, giving earlier incidents first claim on capacity, but it means a stalled earlier incident delays every later payout in the window until the curator advances or cancels it. The stall is always clearable by the operators while the incident is live, since the cancel paths and the selection share the same current and previous period window, and it self clears once the incident's period ages out of that window.

Source Code

The following source code was reviewed during the audit:

- **Repository:** [firelight-core](#)
- **Commit Hash:** d0d373376bdefd97efe5c7a8dd251ac27e12c9b4

Specifically, we audited the following contracts within this repository. Additionally, Firelight made changes after the audit, marked as issues I-2 to I-11, which were reviewed.

Source Code	SHA256
contracts/core/CoverNFT.sol	da52074815a7eec5b418a053d7155992392d3179041b8358137ba819d0084c73
contracts/core/CoverOrderAllocator.sol	ea06ed8b715f60efaea7fe3fd59747e971d09710a1a631f8ff1b24c786f238f6
contracts/core/FirelightVault.sol	4a6d517ae45f7944ce4fb47c82300371f918d5cd34be5bda2baacf6de4420d6
contracts/core/FirelightVaultStorage.sol	95e0038b04608c829ef2f22ef8c219ade2aeba65fd7180abfa01639531c5f0f6
contracts/core/IncidentManager.sol	dc7298ee691143abe7644aef234c5f71e5d67af41c3e1bc4c96afb2cc092abc
contracts/core/VaultRewardDistributor.sol	af91d2b0401b8704593f09a290c11a344510544ce6936e953ed343aa6a805371
contracts/core/interfaces/IAggregatorV3.sol	b3473f120e8a5aa8792807b1c4955ec063398402f8c7cb34daf0f57e629fde2d
contracts/core/interfaces/ICoverOrderAllocator.sol	6b7022a6cfd0fc25e90bc4ac26330d64449c43c82fa94ade53e141b1dd9de8ff
contracts/core/interfaces/IFirelightVault.sol	ec5b25f6bf5516559f8b72c4f5b22fede37af6de8e77a7071a7a133007b0bc2e
contracts/core/interfaces/IIncidentManager.sol	5c9eaa0ff03a284c0c9265f8520e7af82b95ac5f1dc52d8159f30781ef83cb594
contracts/core/lib/Checkpoints.sol	a77a6f3fc67f0b16d402c2ac6a50324f39a1164904995db59f10b36dd03df900
contracts/core/lib/Decimals.sol	01f898306283631b28cee3edca987e151d98d24de125413db58c9a909c063e53
contracts/core/lib/PriceFeed.sol	d0a980eebb9f8c0703845d06dc7cf717b62201366ed058d4d88426734c2d4122
contracts/oracle/FtsoChainlinkAdapter.sol	56d797e28be1d802a04424d2636412dc9446faec1fe0f86e136a935cee6906d8

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

[M-1 Unbounded effectiveLeverage lets real backing approach zero and nullifies the minCAR floor](#)

[M-2 Atomic batchSettle with initial pinned values can DoS runner settlement](#)

[L-1 deposit can mint zero shares while still transferring assets](#)

[L-2 Lowering a concentration cap mid period can strand already created orders](#)

[L-3 setSettlementGracePeriod has no upper bound](#)

[L-4 Code enforces a CAR floor of 1.0 while the specification requires 1.2](#)

[G-1 payout scans periodConfigurations twice for data derivable in one pass](#)

[I-1 Cover NFTs are freely transferable but carry no on-chain rights](#)

[I-2 Add events to Firelight functions](#)

[I-3 Add checkpoint on reward distributor](#)

[I-4 Added function to cancel an allocation commitment](#)

[I-5 Emit deposit limit event during FirelightVault initialize](#)

[I-6 Emit PayoutReceiverUpdated at IncidentManager initialize](#)

[I-7 Enforce minCAR upper bound](#)

[I-8 Bytecode optimizations at CoverOrderAllocator](#)

[I-9 Added matchingCapacity at commitAllocation](#)

[I-10 Cover Settlement - Premium Validation Gap](#)

[I-11 Updated wrong comments](#)

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:
 - How bad things can get (for a vulnerability)
 - The significance of an improvement (for a code quality issue)
 - The amount of gas saved (for a gas optimization)
2. The high/medium/low **likelihood** of the issue:
 - How likely is the issue to occur (for a vulnerability)
3. The overall critical/high/medium/low **severity** of the issue.

This third part - the severity level - is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost .
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, <i>or</i> some funds/assets will be lost, <i>or</i> the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albiet not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

M-1 | Unbounded `effectiveLeverage` lets real backing approach zero and nullifies the `minCAR` floor

Topic: Capacity and solvency | Status: Fixed | Impact: High | Likelihood: Low

`CoverOrderAllocator` sizes period capacity as `collateral × effectiveLeverage / minCAR`, widened by the divergence tolerance. `_setCapacityConfig` validates `minCAR ≥ BPS_DENOMINATOR` (a CAR floor of 1.0) and `effectiveLeverage != 0`, but places no upper bound on `effectiveLeverage`.

```
if (config.minCAR < BPS_DENOMINATOR) revert InvalidMinCAR();
if (address(config.firstLossBufferToken) == address(0)) revert InvalidZeroAddress();
if (config.firstLossBuffer == address(0)) revert InvalidZeroAddress();
// TODO: Add max cap on leverage?
if (config.effectiveLeverage == 0) revert InvalidLeverage();
```

Reference: `CoverOrderAllocator.sol#L604-608`

```
uint256 strictCapacity = availableCollateralCanonical.mulDiv(config.effectiveLeverage, config.minCAR);
// Fold in the divergence tolerance as the period's effective capacity.
totalAvailableCapacity = strictCapacity.mulDiv(BPS_DENOMINATOR + config.divergenceToleranceBps,
BPS_DENOMINATOR);
```

Reference: `CoverOrderAllocator.sol#L661-663`

Because capacity depends on the ratio `effectiveLeverage / minCAR`, the real (un leveraged) collateralization behind sold cover is `minCAR / (effectiveLeverage × (1 + tol))`. The `minCAR` floor of 1.0 bounds only the leveraged CAR, which counts leverage as if it were capital. It does not bound real backing. With `effectiveLeverage` unbounded, an operator can drive real backing arbitrarily low while the leveraged CAR still reads at or above the floor. For example, at `minCAR = 1.0` and leverage of 100x, real backing is roughly 1 percent, yet the on-chain CAR check is satisfied.

The important consequence is that the `minCAR` floor is not an independent solvency protection today. Since leverage sits on top of it with no ceiling, a change to `effectiveLeverage` directly moves the very quantity `minCAR` is meant to guarantee. In effect, unbounded leverage nullifies the `minCAR` protection: the floor becomes real only once leverage is capped. The client has confirmed a maximum will be added (spec: 5.0x absolute ceiling).

Remediations to Consider

- Add a maximum for `effectiveLeverage`. An absolute cap is sufficient to floor real backing only while `minCAR ≥ BPS_DENOMINATOR` remains enforced, so keep that floor as a load bearing invariant.
- A bound expressed relative to `minCAR` (`effectiveLeverage ≤ k × minCAR`) is more robust, since it fixes real backing to `1 / (k × (1 + tol))` independent of the `minCAR` value. Document the guaranteed minimum real backing implied by the chosen bound.

M-2 | Atomic batchSettle with initial pinned values can DoS runner settlement

Topic: Liveness | **Status:** Addressed | **Impact:** Medium | **Likelihood:** Medium

In the allocator runner's `Engine.ts`, `processOrders()` reads the orders and buyer funding at the period start pin block for determinism, reproducing the same tree on every run within the period. Settlement then runs through `EngineRunner.ts` (`settleAll`), which calls `batchSettle()` in the `CoverOrderAllocator`, an atomic loop: one reverting order reverts the whole batch.

A buyer funded at period start who later revokes approval or moves funds makes their `transferFrom` call revert, reverting the entire batch. Because funding is pinned to period start, re-running the matcher yields the same tree and the same revert, so the runner cannot recover; it only warns. A malicious buyer can therefore grief the whole period's batch by approving before the pin and revoking after it. Recovery is manual: recommit a new root excluding the order (allowed while `totalSettledCover == 0`), or settle the fundable orders individually with `settleCoverOrder`.

Remediations to Consider

Consider verifying allowances and balances before settlement and excluding orders without sufficient allowances from the `batchSettle()` parameters, and flagging these orders to be individually settled if clients allow.

Response by Firelight

Addressed in Engine Runner, no contract updates.

L-1 | deposit can mint zero shares while still transferring assets

Topic: Vault accounting | **Status:** Fixed | **Impact:** Low | **Likelihood:** Low

In `FirelightVault.sol`, `deposit()` guards `assets == 0` but not the computed share amount. A small deposit can floor to zero shares while the underlying is still pulled from the depositor; an honest user could lose their deposited assets to rounding.

```
if (assets == 0) revert InvalidAmount();

if (_hasActiveIncident()) revert CurrentPeriodHasActiveIncident();

(uint256 shares, uint256 _totalSupply, uint256 _totalAssets) = _previewTotals(
    assets,
    true,
    Math.Rounding.Floor
);

_depositFunds(_msgSender(), receiver, assets, shares, _totalSupply, _totalAssets);
```

Reference: `FirelightVault.sol#L613-623`

`mint()` function is unaffected since it guards against `shares == 0`. Although the likelihood of small amounts like these being deposited is low, a robustness check could narrow this surface and avoid any dust losses.

Remediations to Consider

- Add `if (shares == 0) revert InvalidAmount();` in `deposit` before `_depositFunds`.

L-2 | Lowering a concentration cap mid period can strand already created orders

Topic: Cover settlement | Status: Acknowledged | Impact: Low | Likelihood: Low

In `CoverOrderAllocator.sol` contract, the settlement grace period is set with no validation.

```
function setSettlementGracePeriod(uint48 newGracePeriod) external onlyRole(CONFIG_ADMIN_ROLE) {
    CoverOrderAllocatorStorage storage $ = _getStorage();
    uint48 oldGracePeriod = $.settlementGracePeriod;
    $.settlementGracePeriod = newGracePeriod;
    emit SettlementGracePeriodUpdated(oldGracePeriod, newGracePeriod);
}
```

Reference: `CoverOrderAllocator.sol`#L553-558

Settlement is blocked until `committedAt + settlementGracePeriod`, and it must also be called while the target period is the current period (`SettleWindowExpired` otherwise). If the grace pushes `graceExpiresAt` to or past the period boundary, there is no instant at which the grace has elapsed and the target period is still current, so no order in that period can settle. All orders stay PENDING, then expire, recoverable via the permissionless expired order sweep. No funds are lost, since the premium is pulled during settlement, but cover issuance halts, and buyers who expected coverage for that period receive none.

Remediations to Consider

- Set a minimum floor for the `settlementGracePeriod()`, ensuring it is shorter than the period duration.
- Consider disallowing `commits` or `recommits` if the remaining period duration is less than the grace period.

Response by Firelight

*Timing confirmed: orders are created during period P with `targetPeriod = P+1`, and concentration updates checkpoint at `currentPeriod() + 1`, so a decrease during P applies exactly to the period where the existing book settles. ****Key property the report understates: the cap is frozen once the settle period starts.**** Any update made during P+1 lands at P+2, and the matcher builds and commits the tree during P+1. If the engine reads `getProtocolConcentrationAt(hash, P+1)` at any point after P+1 begins, it sees the final cap and `ProtocolConcentrationOverflow` is unreachable - affected orders are simply allocated less (PARTIAL) or left unallocated and expire. The settle revert only occurs if the engine matches against a snapshot taken before ****the period start, the same stale-pin pattern as M-2. The proposed remediation (defer decreases to `currentPeriod() + 2`**) is rejected.** It worsens the risk posture: concentration caps are the de-risking lever, and deferring a decrease by a full period extends exposure to a `(chain, protocol)` group precisely when the curator wants to cut it. The current failure mode is the conservative one, less cover is sold on the risky group.*

L-3 | `setSettlementGracePeriod` has no upper bound

Topic: Cover matching | Status: Fixed | Impact: Low | Likelihood: Low

In `CoverOrderAllocator.sol` contract, the concentration caps configuration takes effect at `currentPeriod() + 1`, which is the same period in which orders created now will settle. Lowering a `(chain, protocol)` group cap therefore applies to orders already on the book for that period. At settlement, `_settleCoverOrder()` reads the new lower cap and can revert with `ProtocolConcentrationOverflow`.

The affected orders remain PENDING and are later cancelled, so no premium is charged and no funds are lost. The impact is liveness for those specific orders, and it requires coordination between the curator and the off-chain matching engine to avoid it. The behavior is documented in the contract.

Remediations to Consider

- Defer concentration decreases to `currentPeriod() + 2`, preserving the original cap for orders already created for the next period.

L-4 | Code enforces a CAR floor of 1.0 while the specification requires 1.2

Topic: Spec Breaking | **Status:** Fixed | **Impact:** Low | **Likelihood:** Low

The engineering specification enforces a Capital Adequacy Ratio floor of 1.2 at matching time. The contract only rejects `minCAR` values below `BPS_DENOMINATOR`, which is a floor of 1.0.

```
if (config.minCAR < BPS_DENOMINATOR) revert InvalidMinCAR();
```

Reference: `CoverOrderAllocator.sol`#L604

An operator (or a configuration error) can therefore set `minCAR` between 1.0 and 1.2 and pass validation, sizing capacity above the solvency envelope the specification intends. This is a mismatch between the documented invariant and the enforced one. Combined with M-1, the effective solvency posture is governed entirely by configuration rather than by code.

Remediations to Consider

- If 1.2 is a hard requirement, enforce it in `_setCapacityConfig (minCAR ≥ 1.2e4)`.
- If 1.2 is only a target and 1.0 is the true hard floor, document that explicitly so the discrepancy between spec and code is intentional.

G-1 | payout scans periodConfigurations twice for data derivable in one pass

Topic: Gas Optimization | **Status:** Fixed | **Gas Savings:** Low

In `FirelightVault`, `payout()` function fetches the `capturePeriod` from the `captureTimestamp` and then the `capturePeriodStart` from the `capturePeriod` number:

```
uint256 capturePeriod = periodAtTimestamp(captureTimestamp);
...
uint48 capturePeriodStart = _periodStart(capturePeriod);
```

Reference: `FirelightVault.sol`#L852-L863

```
function periodConfigurationAtTimestamp(uint48 timestamp) public view returns (PeriodConfiguration memory) {
    uint256 length = periodConfigurations.length;
    if (length == 0) revert InvalidPeriod();

    PeriodConfiguration memory periodConfiguration;
    for (uint256 i = 0; i < length; i++) {
        if (timestamp < periodConfigurations[i].epoch)
            break;
        periodConfiguration = periodConfigurations[i];
    }
    if (periodConfiguration.epoch == 0) revert InvalidPeriod();
    return periodConfiguration;
}
```

Reference: `FirelightVault.sol`#L244-L256

However, each scan is $O(n)$ over the period configuration history, which only grows over the vault's life, and the `periodConfigurationAtTimestamp()` used inside `periodAtTimestamp()` already fetches all configuration for the specific timestamp.

```

/**
 * @notice Configuration of a vault period.
 * @param epoch Starting timestamp of this configuration.
 * @param duration Period length in seconds. Must be a multiple of {SMALLEST_PERIOD_DURATION}.
 * @param startingPeriod Period number assigned to `epoch`.
 */
struct PeriodConfiguration {
    uint48 epoch;
    uint48 duration;
    uint256 startingPeriod;
}

```

Reference: IFirelightVault.sol#L27-L37

Consider resolving the timestamp to its PeriodConfiguration once and deriving both the period number and its start timestamp from that single lookup, returning both values in a single loop.

I-1 | Cover NFTs are freely transferable but carry no on-chain rights

Topic: Informational | **Status:** Acknowledged | **Impact:** Informational *

Cover NFTs are standard transferable ERC721 tokens, while claim attribution routes through the order's payoutRecipient, captured at creation and never consulted against the current NFT holder. The specification confirms this is intended (the NFT is a receipt and payouts are routed off-chain to the recorded recipient). The consequence is that transferring or selling a Cover NFT does not transfer coverage: a secondary market buyer of a Cover NFT receives no claim rights.

Response by Firelight

This is intended behaviour.

I-2 | Add events to Firelight functions

Topic: Informational | **Status:** Fixed | **Impact:** Informational *

Added events AddedToBlocklist , RemovedFromBlocklist , AddedToPayoutAllowlist , RemovedFromPayoutAllowlist from FirelightVault contracts.

I-3 | Add checkpoint on reward distributor

Topic: Informational | **Status:** Fixed | **Impact:** Informational *

At VaultRewardDistributor contract, added call to FirelightVault checkpointTotalAssets to update the totalAssets from checkpointed values.

I-4 | Added function to cancel an allocation commitment

Topic: Informational | Status: **Fixed** | Impact: Informational *

Function was added to cancel an allocation commitment.

I-5 | Emit deposit limit event during FirelightVault initialize

Topic: Informational | Status: **Fixed** | Impact: Informational *

Event was added to emit the deposit limit event during FirelightVault initialize.

I-6 | Emit PayoutReceiverUpdated at IncidentManager initialize

Topic: Informational | Status: **Fixed** | Impact: Informational *

Event was added to emit the PayoutReceiverUpdated event at IncidentManager initialize.

I-7 | Enforce minCAR upper bound

Topic: Informational | Status: **Fixed** | Impact: Informational *

The minCAR upper bound was enforced alongside the lower bound.

I-8 | Bytecode optimizations at CoverOrderAllocator

Topic: Informational | Status: **Fixed** | Impact: Informational *

Bytecode optimizations were added to the CoverOrderAllocator contract.

Response by Firelight

Covered in [I-10] remediation via premium calculation.

I-9 | Added matchingCapacity at commitAllocation

Topic: Informational | Status: **Fixed** | Impact: Informational *

Used by the Engine at settling time to generate coverOrder proofs from initial matching commitment.

I-10 | Cover Settlement - Premium Validation Gap

Topic: Informational | Status: **Fixed** | Impact: Informational *

- Full match binds premium exactly, partial accepts any premium $\leq$ total from the leaf with no proportionality check (Contract stores per market rate + cover, the exact premium is computable onchain. The current code instead accepts allocatedPremium from the leaf (only bounding it $\leq$ totalPremiumAmount), making allocatedPremium a trusted leaf field that could be removed entirely).
- Removed premium from merkle tree leaf.

I-11 | Updated wrong comments

Topic: Informational | Status: **Fixed** | Impact: Informational *

Comments were updated.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Firelight team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.